

OfficeMitra (OmERP) — Complete Codebase, Workflow & Sitemap Guide

Target Audience: Anyone (even a 10-year-old child!) can read this file and instantly understand how this multi-tenant SaaS application is coded, how data flows from the web dashboard to the desktop monitoring agent, and what every file and route does.



Table of Contents

1. The Big Picture (What is OfficeMitra?)
 2. End-to-End User Journey & Workflow
 - Phase 1: Company Registration & Onboarding
 - Phase 2: Employee Invitation & Account Setup
 - Phase 3: Transparency Policy Consent
 - Phase 4: Desktop Agent Pairing & Background Tracking
 - Phase 5: Organization Admin Monitoring
 - Phase 6: Platform Super Admin Governance
 3. Complete Sitemap & Route Map
 - A. Public Pages
 - B. Organization Admin & Employee Pages
 - C. Platform Super Admin Console
 - D. Backend API Endpoints
 4. File-by-File Codebase Directory
 - Database Layer (`db/`)
 - Core Libraries & Middleware (`src/lib/`)
 - Desktop Electron Process (`electron/`)
 - Frontend UI & Routes (`src/routes/` & `src/components/`)
 - Automated Tests (`tests/`)
 5. Security & Isolation Guarantees
-

1. The Big Picture (What is OfficeMitra?)

Imagine a giant school building called **OfficeMitra**.

- **The Headmaster (Platform Super Admin):** Owns the whole school building. They can see how many classrooms are rented, how much electricity is being used, and can add or lock classrooms.
 - **Classrooms (Tenants / Organizations):** Each company (like *Acme Corp* or *Stark Industries*) rents one classroom. What happens inside *Acme Corp*'s classroom is **100% secret and invisible** to *Stark Industries*.
 - **The Teacher (Company Owner / Admin):** Runs their own classroom. They can invite students (employees), mark attendance, set working hours, check test results, and manage roles.
 - **The Students (Employees):** Work in their assigned classroom. They log in to their laptop, sign a privacy agreement, and work.
 - **The Robot Assistant (Desktop Electron Agent):** Sits quietly on the student's laptop during school hours (**09:00** to **18:00**). It counts keypresses, measures idle time, and takes helper screenshots. If a student opens a private app (like *WhatsApp* or *Banking*), the Robot Assistant **automatically blurs the image** so nobody can read private messages!
-

2. End-to-End User Journey & Workflow

Phase 1: Company Registration & Onboarding

1. **Visitor Lands:** A business owner opens the website landing page at `/landing`.
2. **Click Register:** They click "*Register Company*" and go to `/register-company`.
3. **Fills Form:** Enters Company Name (*Acme Corp*), Business Email (*sarah@acme.com*), Owner Name (*Sarah Jenkins*), and Password.
4. **Single Transaction Provisioning (`src/lib/store.tsx` & `db/schema.ts`):**
 - Creates a unique `tenant_id` (`t-acme`).
 - Creates the **Owner User** account.
 - Automatically seeds 5 Default Roles: **Owner**, **Admin**, **HR**, **Manager**, **Employee**.
 - Seeds Default Permissions for each role (`employees.view`, `screenshots.view`, `attendance.approve`, etc.).
 - Creates 3 Default Departments: **Engineering**, **Sales & Marketing**, **Human Resources**.
 - Creates Default Productivity Categories (**Productive Tools**: VSCode/GitHub; **Distractions**: YouTube/Twitter).

5. **Email Verification:** Sends a verification link to `sarah@acme.com`.

Phase 2: Employee Invitation & Account Setup

1. **Admin Opens Dashboard:** Sarah logs in at `/login` and views the company dashboard at `/`.
2. **Invites Employee:** Sarah goes to `/employees` or `/invite` and enters an employee's details (*Alex Chen, alex@acme.com, Department: Engineering, Role: Employee*).
3. **Invite Email Sent:** Alex receives an email with a secure token link: `/invite?token=xyz123`.

Phase 3: Transparency Policy Consent

1. **Employee Accepts Invite:** Alex clicks the email link, sets their account password.
2. **Redirection to Consent Screen (`/consent`):** Alex MUST read and accept the monitoring policy before accessing the system.
 - Policy clearly states: Working hours tracking (`09:00-18:00`), screenshot interval (10 mins), automatic Gaussian blur on private apps, zero keystroke content logging.
3. **Consent Recorded (`db/schema.ts`):** Server records `employee_id`, `consent_version: 1`, timestamp, and IP address.
 - *Note:* If the company later updates its consent policy version to `2`, Alex will be prompted to re-consent before desktop tracking resumes!

Phase 4: Desktop Agent Pairing & Background Tracking

1. **Download & Install:** Alex downloads the OfficeMitra desktop app (`electron/main.cjs`).
2. **Agent Login:** Alex logs in using the exact same web credentials (`alex@acme.com`).
3. **OS Secure Credential Storage (`safeStorage`):** The desktop agent encrypts the device token using Windows DPAPI / macOS Keychain (`safeStorage.encryptString(token)`) and saves it to local disk.

4. Background Tracking Loop (`electron/tracker.cjs` & `activity.cjs`):

- **Active Window Polling:** Every 3 seconds (`active-win`), checks active app title ("*VS Code - main.ts*").
- **Main Domain Extraction:** Extracts clean domain ("*github.com*", "*figma.com*").
- **Input Sampling:** Counts total keypresses and mouse movement per minute (`uihook-napi`). Keystroke text is NEVER captured.
- **Gaussian Blur Privacy Protection (`sharp`):** If Alex opens a blacklisted sensitive app (*WhatsApp, Telegram, Banking*), `sharp(shot.buffer).blur(45)` blurs the image 100% unreadable before transmission.
- **Offline Disk Queue:** If internet drops, screenshots save locally to `~/.config/omerp-screenshot-queue/*.json` and auto-drain once reconnected.

Phase 5: Organization Admin Monitoring

1. **Live Activity (`/activity`):** Admin sees which employees are currently working, idle, or offline.
2. **Screenshots Vault (`/screenshots`):** Admin filters by Employee, Department, Date. Views timeline (`09:00, 09:09, 09:18`) with click-to-zoom modal.
3. **Productivity Scores (`/productivity`):** Calculates productive vs unproductive time based on configured app categories.
4. **Workstation Devices (`/devices`):** Admin can view active paired laptops. If a laptop is stolen/lost, clicking "*Revoke Device*" immediately returns `401 Unauthorized` on the next agent request.

Phase 6: Platform Super Admin Governance

1. **Super Admin Access (`/platform`):** Platform Owner logs in as `super_admin`.
2. **SaaS Console View:** The sidebar automatically switches to **Platform Console Mode** showing ONLY SaaS management routes.
3. **Global Metrics:** Views 128 Total Companies, 3,421 Total Employees, 91 Active Devices, 2.3 TB Storage.
4. **Tenant Operations (`/platform/organizations`):** Can activate, suspend, upgrade, or delete any tenant.

5. **Impersonation:** Super Admin can switch context into Tenant A to troubleshoot. Writes an immutable row to `audit_logs` and sets a 60-minute auto-expiring session.
-

3. Complete Sitemap & Route Map

officemitra.io	
└─ 🌐 A. Public Routes (No Login Required)	
└─ /landing	→ Public marketing landing page (Features, Pricing, Security)
└─ /register-company	→ Self-registration form for new companies (Single DB Transaction)
└─ /login	→ Unified login portal for both Web and Desktop Agent
└─ /invite	→ Employee invitation password set page
└─ /consent	→ Employee transparency agreement & consent
signature	
└─ 🏢 B. Organization Admin & Employee Routes (Scoped to Tenant ID)	
└─ /	→ Today's Workforce Overview (Present, Absent, Late, Devices)
└─ /employees	→ Employee Directory & Invite Modal
└─ /leave	→ Workforce Leave Balances & Approval Workflow
└─ /timesheets	→ Attendance Logs & Punch-In / Punch-Out Records
└─ /productivity	→ Productivity Analytics & Category App Rules
└─ /screenshots	→ Screenshot Vault & Interactive Timeline Preview
└─ /screenshots-agent	→ Desktop Capture Agent Setup & Instructions
└─ /departments	→ Company Department Structure
└─ /roles	→ Role-Based Access Control (RBAC) Permission Matrix
└─ /devices	→ Workstations & Revocable Device Tokens
└─ /reports	→ Team & Department Performance Reports
└─ /analytics	→ Employee Productivity Charts & Trends
└─ /my-team	→ Manager's Direct Team Overview
└─ /hr	→ HR Management Console
└─ /settings	→ Company Settings (Screenshot interval, Blur, Working hours)
└─ /billing	→ Tenant Plan Subscription & Point-of-Action Limit Controls
└─ 🦺 C. Platform Super Admin Console (/platform/*)	
└─ /platform	→ Platform Overview (128 Orgs, 3.4k Users, Growth Chart)
└─ /platform/organizations	→ Global Organization Directory & Impersonation Actions
└─ /platform/plans	→ Subscription Plan Tiers & Entitlement Limits
└─ /platform/billing	→ SaaS Financials & Monthly Recurring Revenue (MRR)
└─ /platform/storage	→ Storage Quotas & Presigned Path Validation
└─ /platform/r2	→ Cloudflare R2 Storage Buckets Breakdown
└─ /platform/api-usage	→ Global API Rate Limits & Call Volume Analytics
└─ /platform/feature-flags	→ Tenant Feature Toggles

└─ /platform/settings	→ Global SaaS Infrastructure Settings
└─ /platform/audit-logs	→ Immutable Insert-Only Platform Audit Trail
└─ /platform/support	→ Customer Support Tickets Desk

4. File-by-File Codebase Directory

Database Layer (**db/**)

- **db/schema.ts — Drizzle ORM Relational Schema:**
 - Defines 12 core tables: `tenants`, `tenant_settings`, `departments`, `roles`, `permissions`, `role_permissions`, `employees`, `devices`, `audit_logs`, `impersonation_sessions`, `rate_limits`, `plan_limits`.
- **db/migrations/0000_init_schema.sql — PostgreSQL Migrations & Security Triggers:**
 - `check_role_permission_tenant()` trigger: Ensures role permissions tenant ID matches parent role.
 - `block_audit_log_mutation()` trigger: Prohibits `UPDATE` and `DELETE` on `audit_logs` table (Insert-Only).
 - Row Level Security (RLS) policies enforcing `tenant_id = current_setting('app.current_tenant_id')`.

Core Libraries & Middleware (**src/lib/**)

- **src/lib/tenant.middleware.ts — Connection Pooling Context Guard:**
 - `withTenantContext()`: Wraps database queries inside transaction blocks with `SET LOCAL app.current_tenant_id = $1`.
 - `assertTenantAccess()`: Double-guard helper throwing 403 on cross-tenant attempts.
- **src/lib/permissions.ts — Fixed Permission Catalog & Cache:**
 - `PERMISSION_CATALOG`: Code-defined permissions list (`employees.view`, `screenshots.view`, etc.).
 - `permissionCache`: Tenant-keyed shared cache (`perms:{tenantId}:{roleId}`) with instant invalidation on role edits.
- **src/lib/tenant.ts — Screenshot Paths & R2 Presigning:**
 - `buildR2ScreenshotPath()`: Builds path format `tenant-{tenant_id}/employees/employee-{employee_id}/...`

- `generatePresignedScreenshotUrl()`: Validates tenant ownership before returning presigned short-lived URL.
- `src/lib/store.tsx` — **Client UI State Manager**:
 - Manages reactive UI state, local caching, mock data seeds, and login/register context helpers. (*Note: Security enforcement is handled server-side*).

Desktop Electron Process (`electron/`)

- `electron/main.cjs` — **Master Desktop Agent Process**:
 - Orchestrates screen capture, system tray icon, power suspend auto clock-out, and offline queue.
 - `saveDeviceToken()` / `loadDeviceToken()`: Uses native OS `safeStorage` to encrypt tokens.
- `electron/tracker.cjs` — **Active App & Domain Tracker**:
 - Queries `active-win` for window titles every 3 seconds. Parses clean web domains (*github.com*).
- `electron/activity.cjs` — **Input Sampling Hook**:
 - Uses `uiohook-napi` to sample keypress & mouse counts per minute without text keylogging.

Frontend UI & Routes (`src/routes/` & `src/components/`)

- `src/routes/__root.tsx` — Root application shell, header topbar, login gate, and topbar tenant switcher.
- `src/components/app-sidebar.tsx` — Organization Admin sidebar menu filtered strictly by user permissions.
- `src/components/platform-sidebar.tsx` — Platform Owner Console sidebar menu rendered strictly for `super_admin`.
- `src/routes/roles.tsx` — Interactive Roles & Permission Matrix editor.
- `src/routes/leave.tsx` — Workforce leave balance tracking and approval page.
- `src/routes/platform.tsx` — Parent layout wrapper for Platform Super Admin pages.
- `src/routes/platform.organizations.tsx` — Global tenant directory table with action dropdown (Impersonate, Suspend, Activate, Upgrade, Delete).

Automated Tests (**tests/**)

- **tests/tenant-isolation.test.ts** — Vitest security test suite verifying:
 - Connection pool safety across concurrent requests.
 - Double-guard 403 error on cross-tenant resource access.
 - Cross-tenant R2 screenshot URL rejection.
 - Permission cache invalidation by tenant ID.
 - Unauthenticated session guards.
-

5. Security & Isolation Guarantees

1. **Zero Client Trust:** **tenant_id** is NEVER accepted from client request bodies or headers. It is resolved server-side strictly from verified JWT tokens.
2. **Database-Level RLS Defense:** Even if an application bug omitted a **WHERE tenant_id = X** clause, PostgreSQL Row Level Security rejects rows not belonging to **SET LOCAL app.current_tenant_id**.
3. **Immutable Audit Trail:** The **audit_logs** table has SQL triggers blocking all **UPDATE** and **DELETE** queries at the database engine level.
4. **Revocable Device Tokens:** Revoking a device in the UI immediately updates **status = 'revoked'** in the DB, causing the desktop agent to receive **401 Unauthorized** on its next 3-second ping.
5. **OS Keychain Encryption:** Local desktop agent tokens are stored encrypted using Windows DPAPI / macOS Keychain via Electron's native **safeStorage**.